

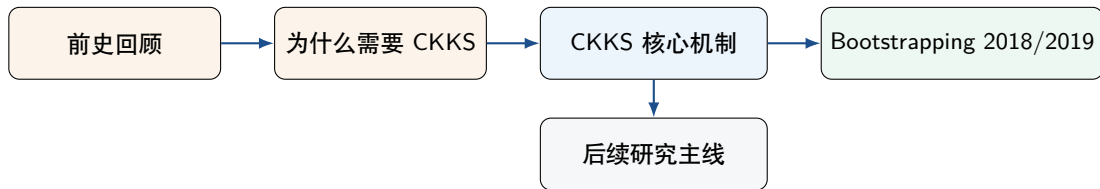
CKKS: Homomorphic Encryption for Arithmetic of Approximate Numbers

From Leveled Approximate HE to Full FHE: Bootstrapping and Subsequent Optimizations

申展

2026 年 4 月 14 日

汇报总路线



要回答的三个问题

- ① 为什么 exact HE 不适合大规模近似数值计算？
- ② 和前面的协议比较，CKKS 到底新在什么地方，尤其是 encoding/approximate decryption/rescaling？
- ③ CKKS 之后的研究主线？——bootstrapping、RNS、evaluator、线性变换与误差管理

一、前史回顾：BGV / BFV 到底解决了什么

方案	解密主语义	乘法后关键步骤	对数值计算的局限
BGV	$\langle c, sk \rangle = m + te \pmod{q}$	relinearization + modulus switching	恢复的是模 t 的精确消息；保的是余数/最低有效位语义
BFV	$\langle c, sk \rangle \approx \Delta m + e$	t/q 缩放 + relinearization	仍然要模 t 的精确消息；fixed-point 位宽会随深度膨胀

共同点

它们都属于 **exact finite-ring arithmetic**：最终目标是恢复一个精确的模消息，而不是恢复一个“带可控误差的实/复数值”。

二、为什么需要 CKKS

核心背景

BGV/BFV 这类 基于 RLWE 的 exact HE 已经支持基于 CRT 的 SIMD 打包：一份明文多项式可以同时装多个槽位并行计算。但是，槽位里的值仍然是有限环中的精确元素；如果想做实数计算，通常要先做 fixed-point 编码

$$x \mapsto \lfloor \Delta x \rfloor,$$

而每做一次乘法，尺度会从 Δ 膨胀到 $\Delta^2, \Delta^3, \dots$ 。如果没有便宜的同态舍入，消息位宽和所需模数都会迅速变大。

显式 CRT-SIMD 例子

$$R_5 = \mathbb{Z}_5[X]/(X^2 + 1), \quad X^2 + 1 = (X - 2)(X - 3) \pmod{5}.$$

由 CRT,

$$m(X) \mapsto (m(2), m(3)) \in \mathbb{Z}_5 \times \mathbb{Z}_5.$$

例如

$$X \leftrightarrow (2, 3), \quad 2 + X \leftrightarrow (4, 0).$$

加法/乘法:

$$X + (2 + X) = 2 + 2X \leftrightarrow (1, 3).$$

$$X(2 + X) = 2X + X^2 \equiv 4 + 2X \leftrightarrow (3, 0).$$

这个例子说明

- 一份明文多项式确实已经能同时装多个槽位，这正是 RLWE 方案中 CRT-SIMD 打包的基本思想。
- 但槽位里的值是 $\mathbb{Z}_5$ 中的元素，而不是 $\mathbb{R}$ 或 $\mathbb{C}$ 中的近似数。
- 所以 BGV/BFV 擅长的是“很多个模值并行算”，不是“近似实数算术”。
- 一旦想做 fixed-point 数值计算，尺度膨胀与同态舍入就会成为核心瓶颈。

三、CKKS 核心机制

主要改进

CKKS 同时改了三件事：

- ① 槽位里：从有限环元素，变成复数近似值；
- ② 解密后恢复：从 exact message，变成 $m + e$ ；
- ③ 乘法后怎么维持数值可解释性：靠 rescaling 主动调整尺度。

复数槽位向量 z

编码为整数多项式 $m(X)$

RLWE 加密

加/乘/relin/rescale

解密 + 解码回槽位

CKKS把 HE 的算术语义改成了 approximate arithmetic。

CKKS 编码/解码：如何将复数向量变成整数多项式

$$z \in \mathbb{C}^{N/2} \xrightarrow{\pi^{-1}} H \xrightarrow{\times \Delta} \Delta \pi^{-1}(z) \xrightarrow{\text{round to } \sigma(R)} \sigma(R) \xrightarrow{\sigma^{-1}} m(X) \in \mathbb{Z}[X]$$

$$\text{Ecd}(z; \Delta) = \sigma^{-1} \left(\lfloor \Delta \cdot \pi^{-1}(z) \rfloor_{\sigma(R)} \right)$$

$$\text{Dcd}(m; \Delta) = \pi \circ \sigma(\Delta^{-1} m)$$

- π^{-1} : 把槽位向量补成**共轭对称**向量;
- σ : 把分圆环多项式看成在若干单位根上的取值;
- Δ : 先把连续复数放大, 避免 rounding 时精度太差;
- $\text{round to } \sigma(R)$: 把连续点拉回离散格点; σ^{-1} : 最终得到**整数系数多项式**。

如何由复数向量 $\rightarrow$ 整数多项式

对分圆环中的实/整数系数多项式 $m(X)$, 若 ζ 是单位根, 则

$$m(\bar{\zeta}) = \overline{m(\zeta)}.$$

也就是说: **一个多项式在共轭单位根上的取值天然成共轭对出现**。因此 CKKS 先把独立的 $N/2$ 个槽位补成一个共轭对称向量 $\pi^{-1}(z)$, 使它正好长成“某个分圆环多项式在所有单位根上的取值”的样子。接着再乘一个大尺度 Δ 并 round 到 $\sigma(R)$ 上, 就把“连续复数几何点”拉回成了“离散的整数系数多项式”。

编码例子 (1): 我们到底想把什么编码进 CKKS

取参数

$$M = 8, \quad \Delta = 64, \quad z = (3 + 4i, 2 - i) \in \mathbb{C}^2.$$

目标

我们真正想装进去的是两个独立槽位:

$$z_1 = 3 + 4i, \quad z_2 = 2 - i.$$

也就是说, 希望找到一个分圆环里的多项式 $m(X)$, 使得它在对应单位根上的取值 “看起来像”

$$(3 + 4i, 2 - i).$$

编码例子 (2): $M = 8$ 时的单位根与共轭结构

令

$$\zeta_8 = e^{2\pi i/8}.$$

4 个相关单位根

$$\zeta_8, \quad \zeta_8^3, \quad \zeta_8^5 = \overline{\zeta_8^3}, \quad \zeta_8^7 = \overline{\zeta_8}.$$

所以它们天然按共轭成对出现:

$$(\zeta_8, \zeta_8^7), \quad (\zeta_8^3, \zeta_8^5).$$

特别地

如果多项式 $m(X)$ 的系数是实数 (这里最后还会 round 成整数), 那么必有

$$m(\bar{\zeta}) = \overline{m(\zeta)}.$$

也就是说: 一个实系数多项式在共轭单位根上的取值, 必然也成共轭对出现。

编码例子 (3): 为什么必须先做“共轭补全”

现在我们要求

$$m(\zeta_8) = 3 + 4i, \quad m(\zeta_8^3) = 2 - i.$$

其余两个位置不能自由选择

由于

$$m(\bar{\zeta}) = \overline{m(\zeta)},$$

所以其余两个位置自动被决定为

$$m(\zeta_8^7) = 3 - 4i, \quad m(\zeta_8^5) = 2 + i.$$

真正用于编码的: 不是 z , 而是 $\pi^{-1}(z)$

按一组自然顺序可写成

$$\pi^{-1}(z) = (3 + 4i, 2 - i, 2 + i, 3 - 4i).$$

这就是“共轭补全”的含义: 用户只给 $N/2$ 个独立槽位, 系统自动补出另外 $N/2$ 个共轭槽位, 使其真能来自某个分圆环多项式的取值。

编码例子 (4): 为什么这个 $\tilde{m}(X)$ 恰好对应目标槽位

论文可取连续版多项式

$$\tilde{m}(X) = \frac{1}{4} (10 + 4\sqrt{2}X + 10X^2 + 2\sqrt{2}X^3).$$

先算 $\tilde{m}(\zeta_8)$

由

$$\zeta_8 = \frac{1+i}{\sqrt{2}}, \quad \zeta_8^2 = i, \quad \zeta_8^3 = \frac{-1+i}{\sqrt{2}},$$

可得

$$4\sqrt{2}\zeta_8 = 4(1+i), \quad 10\zeta_8^2 = 10i, \quad 2\sqrt{2}\zeta_8^3 = 2(-1+i).$$

因此

$$\tilde{m}(\zeta_8) = \frac{1}{4} (10 + 4(1+i) + 10i + 2(-1+i)) = \frac{1}{4} (12 + 16i) = 3 + 4i.$$

编码例子 (5)

再算 $\tilde{m}(\zeta_8^3)$

由

$$\zeta_8^3 = \frac{-1+i}{\sqrt{2}}, \quad (\zeta_8^3)^2 = \zeta_8^6 = -i, \quad (\zeta_8^3)^3 = \zeta_8^9 = \zeta_8 = \frac{1+i}{\sqrt{2}},$$

可得

$$4\sqrt{2}\zeta_8^3 = 4(-1+i), \quad 10(\zeta_8^3)^2 = -10i, \quad 2\sqrt{2}(\zeta_8^3)^3 = 2(1+i).$$

因此

$$\tilde{m}(\zeta_8^3) = \frac{1}{4} \left(10 + 4(-1+i) - 10i + 2(1+i) \right) = \frac{1}{4}(8-4i) = 2-i.$$

如何得到整数多项式

$\tilde{m}(X)$ 还含有 $\sqrt{2}$, 不能直接加密, 所以先乘一个合适的系数, 比如 $\Delta = 64$, 再对系数 rounding, 得到

$$m(X) = 160 + 91X + 160X^2 + 45X^3.$$

解码时, 在相应单位根上求值并除以 64, 得到近似如下, 它已经非常接近于我们的原始槽位了。

$$(3.0082 + 4.0026i, 1.9918 - 0.9974i),$$

CKKS 加密协议 (1): 参数、密钥生成与加密

工作环境与记号

取 power-of-two 分圆环

$$R = \mathbb{Z}[X]/(\Phi_M(X)), \quad R_q = R/qR.$$

设模数链最高层为 q_L 。编码后得到的明文多项式记为

$$m(X) \in R.$$

原论文 concrete construction 中常用的随机分布包括:

- $a \leftarrow R_{q_L}$: 在 R_{q_L} 中均匀采样;
- $e, e', e_0, e_1 \leftarrow DG(\sigma^2)$: 离散高斯噪声;
- $v \leftarrow ZO(0.5)$: 稀疏的 $\{0, \pm 1\}$ 向量;
- s : Hamming weight 为 h 的稀疏秘密向量。

CKKS 加密协议 (1): 参数、密钥生成与加密

KeyGen

秘密钥:

$$sk = (1, s).$$

公钥:

$$pk = (b, a) \in R_{q_L}^2, \quad b = -as + e \pmod{q_L}.$$

evaluation key (供乘法后消掉 s^2 使用):

$$evk = (b', a') \in R_{Pq_L}^2, \quad b' = -a's + e' + Ps^2 \pmod{Pq_L}.$$

Enc

给定明文多项式 $m \in R$, 采样 v, e_0, e_1 , 输出

$$c = (c_0, c_1) = v \cdot pk + (m + e_0, e_1) \pmod{q_L}.$$

也就是

$$c = (vb + m + e_0, va + e_1) \pmod{q_L}.$$

CKKS 加密协议 (2): 解密

Dec

若密文为

$$c = (c_0, c_1) = (vb + m + e_0, va + e_1) \pmod{q_L},$$

则在当前 level ℓ 下解密为

$$\text{Dec}_{sk}(c) = c_0 + c_1 s \pmod{q_\ell}.$$

对新鲜密文可直接看作模 q_L 。

把公钥关系代进去

由

$$b = -as + e \pmod{q_L},$$

可得

$$\begin{aligned} c_0 + c_1 s &= (vb + m + e_0) + (va + e_1)s \\ &= vb + vas + m + e_0 + e_1 s \\ &= v(-as + e) + vas + m + e_0 + e_1 s \\ &= m + (ve + e_0 + e_1 s). \end{aligned}$$

同态加法 vs 同态乘法

加法：语义几乎平稳

若两份密文分别满足

$$\text{Dec}(c_1) = m_1 + e_1, \quad \text{Dec}(c_2) = m_2 + e_2,$$

则

$$\text{Dec}(c_1 + c_2) = (m_1 + m_2) + (e_1 + e_2).$$

原论文中的形式

$$(c_1, \ell, \nu_1, B_1), (c_2, \ell, \nu_2, B_2) \mapsto (c_{\text{add}}, \ell, \nu_1 + \nu_2, B_1 + B_2).$$

结论：

- 消息上界线性增长；
- 噪声上界也线性增长；
- 不会改密文结构，不会让 scale 爆炸。

乘法：立刻出现两类新问题

若

$$\text{Dec}(c_1) = m_1 + e_1, \quad \text{Dec}(c_2) = m_2 + e_2,$$

则

$$\text{Dec}(c_1 \otimes c_2) = (m_1 + e_1)(m_2 + e_2).$$

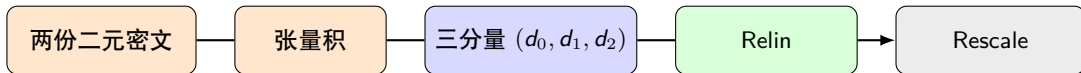
展开得

$$m_1 m_2 + m_1 e_2 + m_2 e_1 + e_1 e_2.$$

因此乘法后同时出现：

- **形状问题**：密文从二分量变成三分量，出现 s^2 ；
- **尺度问题**：若输入 scale 约为 Δ ，主项会变成 Δ^2 。

CKKS 乘法：为什么它会同时带来“形状问题”和“尺度问题”



第一类：形状问题

张量积后

$$(d_0, d_1, d_2) = (b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2)$$

对应的解密形式是

$$d_0 + d_1 s + d_2 s^2.$$

所以乘法先把密文从二分量变成了三分量。

第二类：尺度问题

若输入两份密文都代表 scale 约为 Δ 的消息，则主项从

$$\Delta z_1, \Delta z_2$$

变成

$$\Delta^2 z_1 z_2.$$

所以乘法后必须处理尺度平方膨胀。

乘法后的两步修复：Relinearization 与 Rescaling

Relinearization：修密文形状

目标是把

$$d_0 + d_1s + d_2s^2$$

重新压回标准二分量密文。它依赖 evaluation key 中预存的 Ps^2 信息。

$$c_{\text{mult}} = (d_0, d_1) + P^{-1}d_2 \cdot \text{evk}.$$

结论： Relin 修的是“关于 s^2 的结构问题”，不是 scale。

Rescaling：修数值尺度

若当前密文满足

$$\langle c, sk \rangle = m + e \pmod{q_\ell},$$

则 rescale 后大致变成

$$\langle c', sk \rangle = \frac{q_{\ell-1}}{q_\ell} m + \left(\frac{q_{\ell-1}}{q_\ell} e + e_{\text{scale}} \right).$$

结论： Rescale 修的是“尺度和 level”，同时会丢掉最低有效位。

rescaling 的数值含义：明文缩小、模数下降，但所表示的值基本不变

设当前 level 为 ℓ ，密文满足

$$\langle c, sk \rangle = m + e \pmod{q_\ell},$$

并且它所表示的近似数值可写为

$$x \approx \frac{m}{\Delta}.$$

rescaling 的协议语义

原论文定义

$$RS_{\ell \rightarrow \ell-1}(c) = \left\lfloor \frac{q_{\ell-1}}{q_\ell} c \right\rfloor.$$

因此 rescale 之后，新密文 c' 满足

$$\langle c', sk \rangle = \frac{q_{\ell-1}}{q_\ell} m + \left(\frac{q_{\ell-1}}{q_\ell} e + e_{\text{scale}} \right) \pmod{q_{\ell-1}}.$$

也就是说：

- 模数从 q_ℓ 降到 $q_{\ell-1}$ ；
- 明文多项式与噪声都按同一比例缩小；
- 额外引入一个 rounding 误差 e_{scale} 。

rescaling 的数值含义

一个最小数值例子

若

$$x \approx \frac{m}{\Delta}, \quad m \approx 1234, \quad \Delta = 1000,$$

则当前密文表示

$$x \approx 1.234.$$

做一次按 $p = 10$ 的 rescale 后,

$$m' \approx m/p \approx 123, \quad \Delta' = \Delta/p = 100.$$

因此新密文表示的是

$$x' \approx \frac{m'}{\Delta'} \approx \frac{123}{100} = 1.23.$$

leveled HE

每执行一次 rescaling, 密文就从 level ℓ 降到 level $\ell - 1$ 。因此, 连续乘法会不断消耗模数链, 这正是 CKKS 本质上首先是一个 leveled HE 方案、而后必须借助 bootstrapping 才能走向 fully HE 的原因。

CKKS 不只是 scheme: 作者给出了一套数值函数求解方案

原文中直接展示

- **Power polynomial**: 不断 square, 并在每次 square 后立刻 rescale;
- **General polynomial**: 显式提到可用 Paterson–Stockmeyer;
- **Inverse**: 利用

$$x(1 + \hat{x})(1 + \hat{x}^2) \cdots (1 + \hat{x}^{2^{r-1}}) = 1 - \hat{x}^{2^r}, \quad \hat{x} = 1 - x;$$

- **Exponential / Logistic**: 作为近似函数计算 showcase;
- **FFT / DFT**: 说明 CKKS 天然适合 signal processing 和线性变换。

CKKS 能应用于高维数值函数近似 + 大规模线性变换。

四、为什么 CKKS 需要 bootstrapping

问题的来源

CKKS 是 leveled HE: 每做一次 ciphertext-ciphertext 乘法, 通常都要接一次 rescaling。

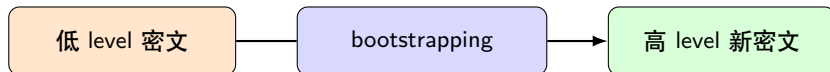
$$q_L \rightarrow q_{L-1} \rightarrow q_{L-2} \rightarrow \dots$$

因此 level 会不断下降, 最终模数链被耗尽, 密文不能继续做深层同态计算。

bootstrapping 的目标

输入: 一份已经快没 level 的 CKKS 密文 ct 。

输出: 一份新的高模数密文 ct' , 仍然表示几乎同一个明文, 但重新获得可继续计算的 level 预算。



为什么要做“系数模归约”

例子

假设低模数 $q = 16$ 下的明文多项式是

$$m(X) = 5 + 3X.$$

这意味着每个系数都是模 16 的余数。因此在更大模数下，同一个对象可以写成

$$t(X) = m(X) + 16l(X),$$

例如

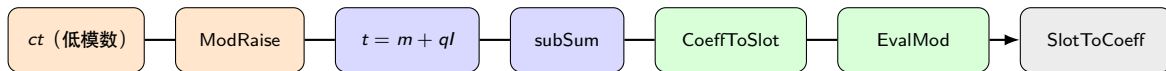
$$t(X) = (5 + 16l_0) + (3 + 16l_1)X.$$

于是 bootstrapping 的目标被改写成

从 $t(X) = m(X) + ql(X)$ 中，近似恢复 $m(X) = [t(X)]_q$.

注意：这里恢复的不是“整个多项式作为一个整体去模 q ”，而是**逐个系数做模归约**。

2018 Bootstrapping 的完整路线图



前半段：把问题变形

- **ModRaise**: 把密文抬到更大模数下看;
- 于是看见“取模前对象”

$$t = m + ql;$$

- **subSum**: 把它整理成适合后续处理的形式。

后半段：真的做模归约

- **CoeffToSlot**: 由系数转换到槽位;
- **EvalMod**: 在槽位里近似恢复 $[t_i]_q$;
- **SlotToCoeff**: 再把结果装回明文多项式。

每个步骤

1. ModRaise

把一份低模数密文提升到更大模数环境下解释。这一步不是“重新加密”，而是为了把原来在模 q 下被隐藏掉的整倍数项重新显露出来，于是可写成

$$t(X) = m(X) + ql(X).$$

2. subSum

对 $t(X)$ 做进一步整理，把“高模数下看到的系数结构”折叠成后续线性变换和模归约更方便处理的对象。

3. CoeffToSlot 与 SlotToCoeff

- **CoeffToSlot**: 把“系数上要做的非线性操作”搬到“槽位里去做”；
- **SlotToCoeff**: 把槽位里恢复好的结果重新拼回打包好的明文多项式。

因为 CKKS 更擅长在槽位上做同态函数评估，所以“系数模归约”必须先挪移到槽位中执行。

真正的难点：EvalMod

前面几步是在把问题搬运和重排；**EvalMod** 是真正要近似恢复 $[t_i]_q$ 的核心步骤。

为 EvalMod 的核心数学 trick

CoeffToSlot 之后，每个槽位里装的是

$$t_i = m_i + ql_i.$$

现在我们要通过 EvalMod 恢复的是

$$m_i = [t_i]_q.$$

第一步：周期性地自动消除 ql_i

因为

$$\sin\left(\frac{2\pi(m_i + ql_i)}{q}\right) = \sin\left(\frac{2\pi m_i}{q}\right),$$

所以未知的整数倍 ql_i 会被正弦函数的周期性自动消掉。

第二步：小角近似把它拉回 m_i

若 m_i 相对 q 足够小，则

$$\sin\left(\frac{2\pi m_i}{q}\right) \approx \frac{2\pi m_i}{q}.$$

因此

$$[t_i]_q \approx \frac{q}{2\pi} \sin\left(\frac{2\pi t_i}{q}\right).$$

EvalMod 中 sine 计算的具体实现

实际实现路线

对于大角度 sine，不是做多项式逼近而是做以下处理：

小角 exponential 的 Taylor 逼近 $\implies$ repeated squaring $\implies$ 恢复大角度 sine.

- ① 先逼近一个更容易处理的小角 exponential；
- ② 再通过 repeated squaring 把角度放大回目标范围；
- ③ 最后从 exponential 中取出 sine 信息，得到 scaled sine。

事实上代价很大：

- repeated squaring 会显著消耗 level；
- 大次数下数值稳定性也不算理想；
- CoeffToSlot / SlotToCoeff 本身是两次很重的线性变换。

因此，2018 bootstrapping 的方法虽然第一次把 CKKS 推到 approximate FHE，但它的瓶颈非常明确：**线性变换** 和 **EvalMod 的实现方式**。

优化对象 1：线性变换内核

对于 bootstrapping 中很重的两步

CoeffToSlot, SlotToCoeff.

不再把它们看成两次笨重的大矩阵乘法，而是把它们改写成 **FFT-like linear transforms**，并进一步利用 **level-collapsing** 在“层数消耗”和“操作数”之间做结构化优化。

优化对象 2：EvalMod 内核

不再走“小角 exponential Taylor 逼近 + repeated squaring”的路线，而是直接在目标区间上做 **Chebyshev approximation of scaled sine**，并为此设计了适配 Chebyshev 基的 **modified Paterson–Stockmeyer** 求值算法。

五、后续主线

主线	代表关键词	一句话脉络
Bootstrapping	EvalMod, high-precision, less modulus	从“补 fully”走向“追求时间 / 精度 / 模数消耗”
RNS	Full-RNS, reduced-error, Grafting	从“搬上机器”走向“重新设计 scale 与 modulus 的关系”
多项式求值	PS, BSGS, lazy-BSGS, asymmetric-BSGS	从“选逼近多项式”走向“HE-aware evaluator tree”
线性变换	FFT-like, hoisting, double-hoisting	从辅助步骤上升为 bootstrapping 与矩阵乘法的主瓶颈
Scale management	EVA, HECATE, ELASM, DaCapo	从手工艺走向编译器自动优化
误差分析	reduced-error, average-case precision	从粗 worst-case 走向实现相关的 precision model

谢谢！

Q&A